

*Islamic University of Technology*

# Lab 4: Amortized Analysis & Greedy Algorithm

CSE 4810: Algorithm Engineering  
**2B**

ASH, ABT(PT)  
June 18, 2026

## General Instruction

This lab consists of two independent parts. Each part contains one **Amortized Analysis** task and one **Greedy Algorithm** task. For every task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file.
- Step 2.** Compile and run the program to produce a data `.txt` file.
- Step 3.** Run the matching Python plotter to generate a chart `.png`.

### Files Provided

|                            |                                                       |
|----------------------------|-------------------------------------------------------|
| <code>Amortized.cpp</code> | Dynamic Table doubling skeleton                       |
| <code>greedy.cpp</code>    | Coin Change greedy-heuristics skeleton                |
| <code>plotter.py</code>    | Plots output of both tasks (auto-detects by filename) |

## Part A: Amortized Analysis of a Dynamic Table

A **dynamic table** (the mechanism behind `std::vector`) starts with some small capacity and *doubles* its capacity whenever it becomes full. The *actual cost* of an `insert()` is 1 (for placing the new element) plus, if a resize is triggered, the number of existing elements that must be copied into the new, larger array.

In the worst case, an insert that triggers a resize on a table of size  $n$  costs  $O(n)$ . This suggests that  $N$  insertions might cost as much as  $O(N^2)$  in total. The **aggregate method** disproves this. Resizes only happen when the table size is a power of two ( $1, 2, 4, 8, \dots$ ), so across  $N$  insertions the total copying work is bounded by a geometric series:

$$1 + 2 + 4 + \dots + N \leq 2N$$

Adding the  $N$  base insertion costs, the total cost of  $N$  insertions is at most  $3N$ , giving an **amortized cost of  $O(1)$  per insertion** — even though individual insertions can spike arbitrarily high.

This produces the same kind of spike-and-average profile you have already seen with the Binary Counter and the Two-Stack Queue: rare, expensive resize operations whose cost is completely absorbed when averaged over the many cheap insertions between them.

## 0.1 TODO 1: Detect a Full Table and Resize

Inside `insert(val)`, before placing the new value:

1. Check if the table is full, i.e. `size == capacity`.
2. If full:
  - (a) Allocate `new_table` with double the capacity (`capacity * 2`).
  - (b) Copy every element from the old `table` into `new_table`.
  - (c) For each element copied, increment `actual_cost` by 1.  
(*This is what makes the resize operation expensive in proportion to its current size.*)
  - (d) `delete[]` the old table, repoint `table` to `new_table`, and update `capacity`.

## 0.2 TODO 2: Insert the New Value

1. Place `val` into `table[size]`.
2. Increment `size`.
3. Return `actual_cost`.

## Sanity Check — Expected Output

```
1 --- SANITY CHECK ---
2 Insert 1: Cost 1 (Expected 1)
3 Insert 2: Cost 2 (Expected 2)
4 Insert 3: Cost 3 (Expected 3)
5 -----
```

**Do not proceed to the plot until all three lines match.** Insert 1 fills capacity 1 with no prior elements to copy (cost 1). Insert 2 finds the table full (size 1 = capacity 1), copies 1 element while resizing to capacity 2, then inserts (cost 1 + 1 = 2). Insert 3 finds the table full again (size 2 = capacity 2), copies 2 elements while resizing to capacity 4, then inserts (cost 1 + 2 = 3).

## Compile & Run

```
1 g++ -O2 -o dyntable Amortized.cpp
2 ./dyntable
3 python3 plotter.py data_dyntable.txt
```

## 0.3 What to Observe in the Plot

- **Blue bars** mark the *actual cost* of every single insertion. Almost every bar has height 1, except for sharp spikes at insertion indices 2, 3, 5, 9, 17, ... — exactly the points where size crosses a power of two and a resize is triggered.

- **Red running-average line** climbs briefly after each early spike, then flattens out and converges toward a small constant as  $N$  grows to 10,000. The early spikes matter a lot when there have only been a few insertions, but their relative contribution vanishes as  $N$  grows.
- **Compare with the Binary Counter and Two-Stack Queue plots** from previous labs. All three share the same amortized-analysis signature: rare, large spikes whose total cost is provably bounded, so the per-operation average stays  $O(1)$ .

## Part B: Greedy Choice Property in Coin Change

The **coin change (minimum coins)** problem asks: given a set of coin denominations and a target value  $V$ , find the minimum number of coins needed to make exactly  $V$  (assuming unlimited supply of each denomination). The standard greedy approach — always take the largest denomination that does not exceed the remaining amount — is fast and simple, but it is only *guaranteed* to be optimal when the coin system has the **greedy choice property** (a so-called **canonical** system, such as  $\{1, 5, 10, 25\}$ ).

You will implement and compare two greedy heuristics across both a canonical and a non-canonical coin system:

| Heuristic                   | Strategy                                                                 | Why It Matters                                                                                                                                            |
|-----------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>greedyStandard</b>       | Always take the largest coin $\leq$ remaining amount                     | <b>Optimal on canonical systems, but blind to structure.</b> On non-canonical systems it can overshoot into a remainder that needs many more small coins. |
| <b>greedyRemainderAware</b> | Looks one step ahead at the <i>remainder</i> before committing to a coin | <b>Tries to avoid bad remainders.</b> A smarter heuristic, but still greedy — it can still fail to find the true optimum on adversarial systems.          |

### 0.4 TODO 1: Implement `greedyStandard(coins, amount)`

1. Loop through the sorted (descending) `coins` vector.
2. While the current `coin` is  $\leq$  the remaining `amount`:
  - (a) Subtract `coin` from `amount`.
  - (b) Increment `count`.
3. Return `count`.

### 0.5 TODO 2: Implement `greedyRemainderAware(coins, amount)`

1. While `amount > 0`, iterate through the available `coins` (largest to smallest) to choose `best_coin`:
  - (a) Consider only coins  $\leq$  `amount`.
  - (b) **Look-ahead logic:** for each candidate coin  $c$ , compute the remainder `amount - c`. If that remainder is evenly divisible by the next smaller denomination in the system, prefer  $c$  over the plain largest-fit choice — this avoids leaving an "awkward" remainder.

- (c) If no candidate satisfies the look-ahead condition, fall back to the largest coin that fits.
2. Subtract the chosen `best_coin` from `amount` and increment `count`.
3. Return `count`.

[Hint] Remove the placeholder `break` once your loop logic is complete — leaving it in will always return after a single iteration and is only there to prevent an infinite loop on the unfinished skeleton.

## Sanity Check — Expected Output

```
1 --- COIN CHANGE SANITY CHECK ---
2 For V=23 (Non-Canonical):
3 Standard Greedy      : 4 coins
4 Remainder-Aware     : 3 coins
5 -----
```

**Do not proceed to the plot until both lines match.** With coins  $\{20, 15, 5, 4, 1\}$  and  $V = 23$ : standard greedy takes  $20 + 1 + 1 + 1 = 23$  (4 coins), overshooting into three wasted 1-coins. The remainder-aware heuristic instead recognizes that  $15 + 4 + 4 = 23$  uses only 3 coins — matching the true optimum for this case.

## Compile & Run

```
1 g++ -O2 -o coins greedy.cpp
2 ./coins
3 python3 plotter.py data_coins.txt
```

## 0.6 What to Observe in the Plot

- **Left chart (Canonical System  $\{1, 5, 10, 25\}$ ):** The green (Standard Greedy) and orange dashed (Remainder-Aware) lines sit *exactly on top of each other* for every value of  $V$  from 1 to 100. On a canonical system, the greedy choice property guarantees the simple heuristic is already optimal — there is nothing for the look-ahead logic to improve.
- **Right chart (Non-Canonical System  $\{1, 4, 5, 15, 20\}$ ):** The crimson (Standard Greedy) and violet dashed (Remainder-Aware) lines **diverge** at several values of  $V$ , with Standard Greedy occasionally needing visibly more coins than Remainder-Aware for the same target value.
- **Think about it:** Even the smarter remainder-aware heuristic is still fundamentally greedy — it commits to one coin at a time and never reconsiders. On sufficiently adversarial coin systems it can still miss the true optimum, which only a full dynamic-programming solution is guaranteed to find. The greedy choice property is a property of the *coin system*, not of how clever your greedy heuristic is.